Shiam Ezzo — Tasks (Backend Questions + Interviews, Test plan, Full-flow test)

Purpose

You build the two content modules users create — **questions** and **interviews** — with owner-only editing, and you own the overall test plan plus the final end-to-end test that proves the whole app works together.

Read first: `docs/SPECIFICATION.md`, `docs/API-CONTRACT.md` (sections 1, 2, 3, 5, 6, 8), `docs/TEAM-OWNERSHIP.md`.

1. Files you own (only you edit these)

File	What goes in it
`backend/src/questions/questions.module.ts`	Question model, controller, service
`backend/src/questions/schemas/question.schema.ts`	Fields from contract §8, indexes
`backend/src/questions/questions.controller.ts`	`GET /questions`, `GET /questions/:id`, `POST`, `PATCH /:id`, `DELETE /:id`; `@ApiTags('questions')`, `@ApiBearerAuth()`
`backend/src/questions/questions.service.ts`	Filters, owner check (403 `NOT_OWNER`), public vs owner view, in-use check before delete, numeric id helper (`counters` key `questions`),

File	What goes in it
	`ownerUsername` lookup from `users` collection
`backend/src/questions/dto/question-filter.dto.ts`	`category`, `difficulty`, `type`, `ownerId`, `search`
`backend/src/questions/dto/create-question.dto.ts`	Common + type-specific validation
`backend/src/questions/dto/update-question.dto.ts`	Partial update; final result must still be valid
`backend/src/questions/dto/question-response.dto.ts`	`PublicQuestion`, `OwnerQuestion` for Swagger
`backend/src/questions/question-validation.ts`	Cross-field rules (exactly one correct choice, keyword count, no fields from other types)
`backend/src/interviews/interviews.module.ts`	Interview model, controller, service
`backend/src/interviews/schemas/interview.schema.ts`	Fields from contract §8, indexes
`backend/src/interviews/interviews.controller.ts`	`GET /interviews`, `GET /interviews/:id`, `POST`, `PATCH /:id`, `DELETE /:id`
`backend/src/interviews/interviews.service.ts`	Validate `questionIds` exist, build detail with public questions in order, owner checks, numeric id helper (`counters` key `interviews`)
`backend/src/interviews/dto/interview-filter.dto.ts`	Query validation

File	What goes in it
`backend/src/interviews/dto/create-interview.dto.ts`	1–20 distinct ids, time limit 30–3600
`backend/src/interviews/dto/update-interview.dto.ts`	Partial update
`backend/src/interviews/dto/interview-response.dto.ts`	`InterviewSummary`, `InterviewDetail`
`backend/src/questions/seed/seed-questions.ts`	At least 40 team-authored questions, 4+ categories, all difficulties, all 3 types, owned by a seed user
`backend/src/questions/seed/seed.command.ts`	Idempotent seed script (running twice doesn't duplicate) — creates the seed user directly in `users` with the contract shape if missing
`backend/test/questions.e2e-spec.ts`	Your module tests
`backend/test/interviews.e2e-spec.ts`	Your module tests
`backend/test/full-flow.e2e-spec.ts`	Whole-app test (written now, run on integration day)
`backend/test/helpers/fake-auth.guard.ts`	Fake guard for **your** tests (others write their own copy)
`docs/TEST-PLAN.md`	Scope, acceptance matrix, test data, pass/fail status
`README.md` → sections *Questions and interviews*, *Testing*	Only these headings
`backend/src/app.module.ts`	**Only** add `QuestionsModule` and `InterviewsModule`

File	What goes in it
	import lines (coordination §3)

2. Files you must NOT edit

- `backend/src/auth/`, `users/`, `main.ts`, the rest of `app.module.ts` (Badr), `backend/src/attempts/` (Esmaeel), all `frontend/`.
- You **read** the `users` and `attempts` collections (for `ownerUsername` and the in-use check) but never write to them.
- Don't change production code of other modules to make a test pass — report the bug to the owner with steps to reproduce.

3. Implementation order

1. Question schema + DTOs + cross-field validation. Unit-test the validation rules first (they're the hardest part).
2. Questions controller/service with the fake guard in tests. Test owner vs non-owner views.
3. Delete-in-use check: reads `interviews` (your own) **and** `attempts` (`questions.questionId`). In tests, insert a raw attempt document.
4. Interview schema + DTOs + service. Detail returns public questions in order and skips deleted ones.
5. Seed script (questions are owned by a seed user, e.g. username `prepbot`).
6. `docs/TEST-PLAN.md`: acceptance matrix from the spec's definition of done, one row per rule, owner column, status column.
7. `full-flow.e2e-spec.ts`: register two users through the real HTTP API → user A creates questions + interview → user B can read, gets 403 on edit → B takes the interview, answers, finishes, reads results → profile stats updated → A edits a question → B's old results unchanged. Write it now against the contract; it will pass on integration day.

4. Things to think about

- PATCH that changes `type` from `multiple_choice` to `short_answer`: what should happen to the old `choices`?
- Why is the interview detail always public-form even for the owner?
- Seed data owner: how do you make the seed idempotent without duplicating the seed user or the questions?
- How does `ownerUsername` stay correct without `populate`? (One `find` on `users` with `$in` for all owner ids.)

5. Completion checklist

- [ ] Non-owner PATCH/DELETE → 403 `NOT_OWNER`; owner succeeds.
- [ ] Answer keys only in owner view; never in `/interviews/:id`.
- [ ] MC: 2–6 choices, exactly one correct. Short answer: 2–6 keywords, valid required count. Code output: expected output required.
- [ ] Fields of another type rejected; unknown fields rejected.
- [ ] Interview with 0 or 21 questions rejected; duplicate ids rejected; unknown id → 400 `QUESTION_NOT_FOUND_IN_LIST`.
- [ ] Used question → 409 `QUESTION_IN_USE` (both "in an interview" and "in an attempt" cases).
- [ ] Seed: ≥40 questions, ≥4 categories, all difficulties, all types; second run adds nothing.
- [ ] Swagger complete for both modules.
- [ ] TEST-PLAN.md filled and updated after integration day.